IPA 1/2569

HCL Components & First AWS Deployment

init

plan

apply

WEEK 11

Week 1 Agenda

3 hours: ~60 min concept + ~120 min hands-on lab

- Quick Recap: IaC & Terraform flow
- HCL block syntax deep dive
- Terraform settings block
- Providers — registry, config, aliases
- Resources & data sources
- Meta-arguments overview
- Modules & import preview

LAB: Provision EC2 on AWS

- provider.tf, main.tf setup
- AMI data source, instance tags
- init → fmt → plan → apply

Quick Recap: IaC & Terraform Flow

Assumed from prior intro — brief review

Infrastructure as Code

- Declarative model: describe desired state, Terraform figures out HOW to get there
- Idempotent: apply the same config repeatedly → same result
- Version-controlled, reviewable, repeatable
- AWS focus: provider handles API calls for you

Terraform Deployment Flow

terraform init — download providers & modules
terraform fmt — auto-format HCL code
terraform plan — preview changes (dry run)
terraform apply — execute & update state
terraform destroy — tear down all resources

terraform init
Download providers

→

terraform fmt
Format code

→

terraform plan
Preview changes

→

terraform apply
Create resources

State file (terraform.tfstate) tracks real-world resource IDs after every apply

HCL Block Syntax

The building block of every Terraform configuration | Ch. 2.2

```
<block_type>  "<TYPE>"  "<NAME>"  {

  argument_name = "value"

  nested_block {

    sub_argument = true

  }

}

# Example: resource block

resource "aws_instance" "web" {

  ami           = data.aws_ami.al2.id

  instance_type = var.instance_type

}
```

Block Type	resource, data, variable, output, locals, provider, terraform, module
Type Label	Provider resource type (e.g. aws_instance). Required for resource & data.
Name Label	Local name to reference this block. Required for resource & data.
Arguments	Key = value pairs. Values can be strings, numbers, bools, lists, maps.
Subblocks	Nested { } blocks for grouped settings (lifecycle, tags, ingress…).
Ordering	HCL is declarative — block order does NOT matter (Terraform resolves deps).

Terraform Settings Block

Global configuration for the Terraform run | Ch. 2.3

```
terraform {
  required_version = ">= 1.6"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {          # Week 5
    bucket = "my-tf-state"
  }
}
```

required_version

Constrains which Terraform CLI versions may run this config. Prevents accidental upgrades breaking syntax.

required_providers

Declares provider source (registry path) and version constraints. Needed so init knows what to download.

backend block

Where state is stored. Default = local file. S3/Azure/GCS for teams. Covered fully in Week 5.

cloud block

Alternative to backend — connects to HCP Terraform / Terraform Cloud. Not covered in this course.

experiments

Opt-in to preview language features (e.g. module_variable_optional_attrs). Rarely needed in practice.

Providers

The bridge between Terraform and external APIs | Ch. 2.4

Provider Registry

registry.terraform.io hosts all providers

hashicorp/aws, hashicorp/azurerm, etc.

Declared in required_providers block

Downloaded by terraform init

Provider Configuration

```
provider "aws" {
  region  = var.aws_region
  profile = "dev"
}
```

Can use env vars: AWS_REGION etc.

Provider Aliases

Multiple configs of same provider

```
provider "aws" {
  alias  = "us_east"
  region = "us-east-1"
}
```

```
Reference: provider = aws.us_east
```

💡 Best practice: pin provider version with ~> (pessimistic constraint) so patch updates install but major breaking changes are blocked. Use environment variables or AWS profiles for credentials — never hardcode keys in .tf files.

Resources & Data Sources

Creating and reading infrastructure | Ch. 2.5–2.6

Resources — what Terraform creates / manages

```
resource "aws_instance" "web" {
  ami           = "ami-0abc123"
  instance_type = "t3.micro"
  tags = { Name = "web-server" }
}
```

- Address: resource.<TYPE>.<NAME> e.g. resource.aws_instance.web
- Read attribute: aws_instance.web.id or aws_instance.web.public_ip
- Terraform manages full lifecycle: create → update → destroy
- Multiple resource types from same provider (aws_vpc, aws_sg, …)

Data Sources — read existing / external info

```
data "aws_ami" "al2" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["amzn2-ami-hvm-*"]
  }
}
```

- Address: data.<TYPE>.<NAME> e.g. data.aws_ami.al2
- Read-only — Terraform never creates or changes the data source
- Use .id to reference: ami = data.aws_ami.al2.id
- Refreshed on every plan unless ignore_changes is set

Meta-Arguments

Special arguments that control resource behaviour | Ch. 2.7

depends_on

Explicit dependency when Terraform can't auto-detect it.
Forces one resource to wait for another.

```
resource "aws_instance"
"app" {
  depends_on =
[aws_db_instance.db]
}
```

provider

Override which provider config handles this resource.
Used with aliased providers for multi-region setups.

```
resource "aws_instance" "eu"
{
  provider = aws.eu_west
}
```

lifecycle

Control create/update/destroy behaviour:

- create_before_destroy
- prevent_destroy
- ignore_changes

```
lifecycle {
  create_before_destroy =
true
  prevent_destroy       =
true
}
```

count & for_each

Create multiple instances of a resource.
count = integer, for_each = set/map.
Covered in depth in Week 3.

```
resource "aws_instance"
"web" {
  count = 3
  ami   = var.ami_id
}
```

Week 1 — Key Takeaways

What you should be able to do after today

HCL Anatomy

Every Terraform file is made of blocks. You know block types, labels, arguments, and nested subblocks.

Terraform Settings

The terraform {} block pins required_version and required_providers — always include it.

Providers

Providers are plugins. Declare them, configure credentials safely, and pin versions with ~>.

Resources vs Data Sources

resource creates; data reads. Both addressable by type + name. Data is read-only.

Meta-Arguments

depends_on for explicit order, lifecycle for create/destroy control, provider for aliases.

Full Workflow

init → fmt → plan → apply → destroy. State file is the source of truth between runs.

Next week: Variables, locals, outputs, type system & building your first reusable module → Ch. 3